



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ _____ «Информатика и системы управления»

КАФЕДРА _____ «Теоретическая информатика и компьютерные технологии»

РАСЧЕТНО-ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
К ВЫПУСКНОЙ КВАЛИФИКАЦИОННОЙ РАБОТЕ
НА ТЕМУ:
«Здесь пишем тему»

Студент _____
(Группа)

(Подпись, дата)

(И.О. Фамилия)

Руководитель ВКР

(Подпись, дата)

(И.О. Фамилия)

Консультант

(Подпись, дата)

(И.О. Фамилия)

Консультант

(Подпись, дата)

(И.О. Фамилия)

Нормоконтролер

(Подпись, дата)

(И.О. Фамилия)

2026 г.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	4
1 Исследовательская часть	5
1.1 Интуиционистская логика	5
ВНК-интерпретация	6
Семантика Крипке	7
1.2 Системы вывода	9
Система Гильберта	9
Натуральный вывод	10
Секвенциальное исчисление	11
1.3 Типизированное λ -исчисление	14
Аннотирование правил LJ (импликативный фрагмент)	14
Суперскрипты для произведения и суммы	15
Аннотирование правил LJ (полный фрагмент)	16
2 Конструкторская часть	17
2.1 Исчисление LJТ	17
Исчисление LJ в формулировке Дайкхоффа	17
Проблема завершаемости	18
Правила исчисления LJТ	18
Обратный поиск вывода	19
Пример вывода	20
2.2 Исчисление LJT_{SAT}	20
Клаузальная форма	20
Правила исчисления LJT_{SAT}	22
Процедура поиска вывода	23
Построение контрмодели Крипке	24
2.3 Аннотирование термами	24
Аннотирование правил LJТ	25
Клауфикация как секвенциальные правила	25
Извлечение терма из клаузальной формы	27
2.4 Исчисление $C^{\rightarrow}$	27

Правила исчисления	28
Аннотирование правил	28
Процедура поиска вывода	29
Преимущества перед LJT_{SAT}	30
ЗАКЛЮЧЕНИЕ	31
СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ	32
ПРИЛОЖЕНИЕ А	33
ПРИЛОЖЕНИЕ Б	34
ПРИЛОЖЕНИЕ В	35

ВВЕДЕНИЕ

Изоморфизм Карри-Говарда устанавливает удивительное соответствие между системами формальной логики и вычислительными. Все начинается с наблюдения о том, что импликация $A \rightarrow B$ соответствует типу функции принимающей значение типа A и возвращающей значение типа B . Так как вывод формулы B из посылок $A \rightarrow B$ и A может рассматриваться как применение первой посылки ко второй, точно так будто функция из A в B примененная к A возвращает B .

Данная идея – сопоставление логических формул и некоторых вычислительных выражений – является довольно богатой идеей. Благодаря ней развивается очень перспективная сфера - формальная верификация программ. Формальная верификация позволяет провалидировать (верифицировать) критически важные программы (или их компоненты), например, в ПАО «Группа Астра», занимающейся разработкой ОС Astra Linux, модуль РАМ (управление привелигированным доступом) должен быть формально верифицирован, т.е. должна быть доказана корректность как самих алгоритмов так и их реализации.

Изучение изоморфизма Карри-Говарда сопряжено с существенными трудностями. Прежде всего, сложность обусловлена необходимостью одновременного понимания нескольких абстрактных областей: математической логики (в частности, интуиционистской логики), λ -исчисления и типовых систем. Каждая из этих областей сама по себе требует серьезной теоретической подготовки, а их объединение в рамках единого соответствия «доказательства $\leftrightarrow$ программы» и «формулы $\leftrightarrow$ типы» создает высокий порог входа. Дополнительную сложность вносит высокий уровень абстракции: многие соответствия не имеют наглядной интерпретации и воспринимаются исключительно символически, что затрудняет формирование интуитивного понимания.

В связи с этим разработка специализированного визуализатора представляется особенно актуальной. Создание системы, позволяющей интерактивно отображать соответствия между логическими доказательствами и программами, может существенно повысить доступность данной темы, упростить процесс обучения и способствовать более глубокому пониманию фундаментальных принципов, лежащих в основе современных типовых систем и методов формальной верификации.

1 Исследовательская часть

1.1 Интуиционистская логика

Интуиционистская логика возникла в контексте кризиса оснований математики — масштабного пересмотра фундаментальных допущений, на которых строилось математическое знание. Во второй половине XIX века Георг Кантор заложил основы теории множеств, предложив рассматривать бесконечные совокупности как полноправные математические объекты. Теория Кантора позволила единообразно описать числовые системы, точки континуума и функциональные пространства, однако вскоре обнаружились серьёзные затруднения: в 1897 году Чезаре Бурали-Форти указал на парадокс, связанный с множеством всех ординалов, а в 1901 году Бертран Рассел сформулировал знаменитый парадокс о множестве всех множеств, не содержащих самих себя. Эти противоречия показали, что наивное обращение с бесконечными объектами чревато антиномиями и что сама система допущений классической математики нуждается в критическом анализе.

В ответ на кризис сформировались несколько программ обоснования математики. Давид Гильберт предложил формалистический подход: зафиксировать аксиомы и правила вывода, а затем доказать непротиворечивость полученной формальной системы финитными средствами. Готлоб Фреге и Бертран Рассел развивали логицистскую программу, стремясь свести математику к логике. Третий путь избрал нидерландский математик Луитзен Эгбертус Ян Брауэр: его интуиционизм, оформившийся в 1907–1912 годах, предполагал радикальный пересмотр не только оснований, но и самих методов математического рассуждения.

Брауэр исходил из того, что математические объекты не существуют независимо от познающего субъекта, а являются мысленными конструкциями. Математическое утверждение считается истинным лишь тогда, когда для него предъявлена конструкция — явная процедура, позволяющая убедиться в его справедливости. Следствием этой позиции стало отвержение закона исключённого третьего (*tertium non datur*): высказывание $A \vee \neg A$ не может приниматься как логический закон, поскольку для произвольного A мы можем не располагать ни доказательством A , ни доказательством $\neg A$. Классическая логика, напротив, постулирует этот закон безусловно, допуская тем самым неконструктивные доказательства су-

существования: утверждение $\exists x. P(x)$ считается доказанным, даже если конкретный x не предъявлен, а лишь показано, что отсутствие такого x ведёт к противоречию.

Именно неконструктивность являлась главной мишенью критики Брауэра. Классическое доказательство может гарантировать существование объекта, не указывая ни способа его построения, ни алгоритма нахождения. Для Брауэра такое доказательство лишено математического содержания: оно не даёт конструкции и потому не может считаться обоснованием истинности. Отказ от закона исключённого третьего и, как следствие, от снятия двойного отрицания ($\neg\neg A \rightarrow A$), закона Пирса ($((A \rightarrow B) \rightarrow A) \rightarrow A$) и ряда других классически допустимых принципов позволил ограничить доказательства лишь теми, которые несут конструктивное содержание.

Формальная аксиоматизация интуиционистской логики была осуществлена учеником Брауэра — Арендом Гейтингом — в 1930 году. Гейтинг предложил исчисление высказываний, в котором каждая аксиома и правило вывода допускают конструктивную интерпретацию, а закон исключённого третьего не выводим.

ВНК-интерпретация

В 1930–1945 годах Андрей Николаевич Колмогоров и независимо Гейтинг сформулировали интерпретацию логических связок через задачи и решения, известную как ВНК-интерпретация (Брауэр – Гейтинг – Колмогоров). Семантика связок в интуиционистской логике определяется через понятие доказательства:

- **Конъюнкция** $A \wedge B$ считается доказанной, если имеются доказательства как A , так и B ;
- **Дизъюнкция** $A \vee B$ доказана, если известно, какая из частей истинна, и имеется её доказательство;
- **Импликация** $A \rightarrow B$ интерпретируется как конструкция (метод), преобразующая любое доказательство A в доказательство B ;
- **Отрицание** $\neg A$ определяется как $A \rightarrow \perp$, где $\perp$ обозначает ложь;
- **Ложь** $\perp$ не имеет доказательства.

ВНК-интерпретация придала интуиционистской логике ясный вычислительный смысл и послужила одной из отправных точек для изоморфизма Карри–Говарда, устанавливающего соответствие между доказательствами и программами.

Семантика Крипке

Семантика Крипке задаёт класс моделей, в которых истинность формулы определяется относительно миров, связанных отношением частичного порядка. Формула называется общезначаимой, если она истинна в корне любой модели. Контрмодель — модель, в корне которой некоторая формула не вынуждена; существование контрмодели показывает, что формула не является теоремой интуиционистской логики.

Моделью Крипке называется четвёрка $(W, w_0, \preceq, V)$, где

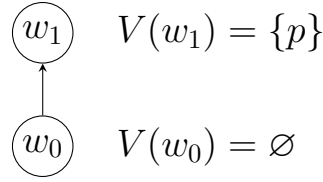
- W — непустое конечное множество миров;
- $w_0 \in W$ — выделенный корневой мир, из которого достигим любой другой: $w_0 \preceq w$ для всех $w \in W$;
- $\preceq$ — частичный порядок (рефлексивное, антисимметричное и транзитивное отношение) на W ;
- V — оценка, сопоставляющая каждой пропозициональной переменной p множество миров $V(p) \subseteq W$, в которых p истинна; оценка удовлетворяет условию наследуемости (монотонности): если $w \in V(p)$ и $w \preceq v$, то $v \in V(p)$.

Отношение вынуждения $w \Vdash A$ (« A истинна в w ») определяется индуктивно:

- $w \Vdash p$ тогда и только тогда, когда $w \in V(p)$;
- $w \Vdash A \wedge B$ тогда и только тогда, когда $w \Vdash A$ и $w \Vdash B$;
- $w \Vdash A \vee B$ тогда и только тогда, когда $w \Vdash A$ или $w \Vdash B$;
- $w \Vdash A \rightarrow B$ тогда и только тогда, когда для всякого $v \succeq w$ из $v \Vdash A$ следует $v \Vdash B$;
- $w \Vdash \perp$ никогда не выполняется.

Тогда $w \Vdash \neg A$ эквивалентно тому, что для всех $v \succeq w$ имеем $v \nVdash A$. Из определения следует монотонность вынуждения: если $w \Vdash A$ и $w \preceq v$, то $v \Vdash A$. Формула в интуиционистской логике общезначаима, если $w_0 \Vdash A$ в любой модели и любой оценке

Пример 1. Рассмотрим цепочку из двух миров $w_0 \prec w_1$ и одну переменную p , положив $V(p) = \{w_1\}$ (наследуемость соблюдена).

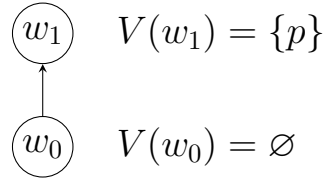


Тогда $w_1 \Vdash p$, а $w_0 \nVdash p$.

Закон исключённого третьего $p \vee \neg p$. В мире w_0 не вынуждена ни p (нет вынуждения в w_0), ни $\neg p$ (так как $w_1 \succeq w_0$ и $w_1 \Vdash p$). Следовательно, $w_0 \nVdash p \vee \neg p$. Данная модель является контрмоделью, и закон исключённого третьего не общезначим в интуиционистской логике.

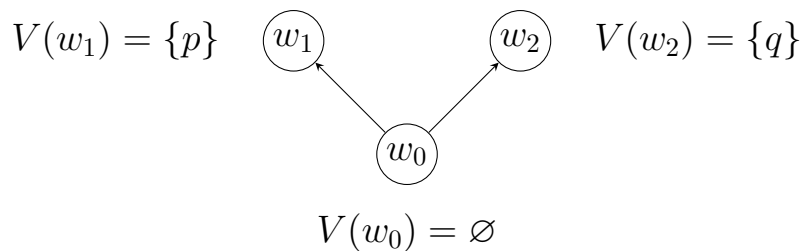
Схема снятия двойного отрицания $\neg\neg p \rightarrow p$. Та же модель служит контрмоделью. Имеем $w_0 \nVdash p$. При этом $w_0 \Vdash \neg\neg p$: для любого $v \succeq w_0$, если бы $v \Vdash \neg p$, то для всех $u \succeq v$ было бы $u \nVdash p$, что неверно при $v = w_1$. Значит, $v \nVdash \neg p$ для всех $v \succeq w_0$, откуда $w_0 \Vdash \neg\neg p$. Итак, $w_0 \Vdash \neg\neg p$, но $w_0 \nVdash p$, поэтому $w_0 \nVdash \neg\neg p \rightarrow p$.

Пример 2. Закон Пирса $((p \rightarrow q) \rightarrow p) \rightarrow p$. Возьмём ту же цепочку $w_0 \prec w_1$, но с двумя переменными: $V(p) = \{w_1\}$, $V(q) = \emptyset$.



Тогда $w_1 \nVdash q$, откуда $w_0 \nVdash p \rightarrow q$: для $v = w_1$ имеем $v \Vdash p$, но $v \nVdash q$. Проверим $w_0 \Vdash (p \rightarrow q) \rightarrow p$: для любого $v \succeq w_0$ нужно «если $v \Vdash p \rightarrow q$, то $v \Vdash p$ ». При $v = w_0$ посылка $w_0 \Vdash p \rightarrow q$ ложна, импликация выполняется. При $v = w_1$ посылка $w_1 \Vdash p \rightarrow q$ требует, чтобы для всех $u \succeq w_1$ из $u \Vdash p$ следовало $u \Vdash q$; при $u = w_1$ это даёт $w_1 \Vdash q$, что неверно, значит $w_1 \nVdash p \rightarrow q$, и импликация тривиальна. Следовательно, $w_0 \Vdash (p \rightarrow q) \rightarrow p$, но $w_0 \nVdash p$, поэтому $w_0 \nVdash ((p \rightarrow q) \rightarrow p) \rightarrow p$.

Пример 3. Не всякая контрмодель является цепочкой. Рассмотрим формулу $(p \rightarrow q) \vee (q \rightarrow p)$ и модель с ветвлением:



Здесь $w_0 \not\models p \rightarrow q$, поскольку $w_1 \models p$, но $w_1 \not\models q$. Аналогично $w_0 \not\models q \rightarrow p$, так как $w_2 \models q$, но $w_2 \not\models p$. Следовательно, $w_0 \not\models (p \rightarrow q) \vee (q \rightarrow p)$. Эта формула является аксиомой логики Гёделя–Даммита, но не теоремой интуиционистской логики.

Интуиционистская логика играет фундаментальную роль в изоморфизме Карри–Говарда, поскольку именно её конструктивная природа позволяет установить соответствие между логическими доказательствами и программами. В рамках данного соответствия логические формулы интерпретируются как типы, а доказательства — как программы, что делает интуиционистскую логику естественной логической основой типизированных функциональных языков программирования.

1.2 Системы вывода

Формальные системы вывода представляют собой строгие математические конструкции, предназначенные для задания правил построения корректных доказательств в рамках заданной логики. Ниже рассматриваются три основных подхода: аксиоматическая система в стиле Гильберта, натуральный вывод и секвенциальное исчисление.

Система Гильберта

Гильбертова система представляет собой аксиоматическую формализацию интуиционистской логики. Она включает набор аксиом и правило вывода *modus ponens*:

$$\frac{A \quad A \rightarrow B}{B}$$

Ниже приведён стандартный набор аксиом:

- (A1) $A \rightarrow (B \rightarrow A)$
- (A2) $(A \rightarrow (B \rightarrow C)) \rightarrow ((A \rightarrow B) \rightarrow (A \rightarrow C))$
- (A3) $A \wedge B \rightarrow A$
- (A4) $A \wedge B \rightarrow B$
- (A5) $A \rightarrow (B \rightarrow A \wedge B)$
- (A6) $A \rightarrow A \vee B$
- (A7) $B \rightarrow A \vee B$
- (A8) $(A \rightarrow C) \rightarrow ((B \rightarrow C) \rightarrow (A \vee B \rightarrow C))$
- (A9) $\perp \rightarrow A$

Пример. Вывод формулы $A \rightarrow A$:

- 1. $A \rightarrow ((A \rightarrow A) \rightarrow A)$ (A1)
- 2. $(A \rightarrow ((A \rightarrow A) \rightarrow A)) \rightarrow ((A \rightarrow (A \rightarrow A)) \rightarrow (A \rightarrow A))$ (A2)
- 3. $(A \rightarrow (A \rightarrow A)) \rightarrow (A \rightarrow A)$ (MP 1, 2)
- 4. $A \rightarrow (A \rightarrow A)$ (A1)
- 5. $A \rightarrow A$ (MP 4, 3)

Гильбертова аксиоматика компактна и удобна для металогических рассуждений, но плохо приспособлена для практического построения доказательств: даже простые формулы задаются длинными цепочками подстановок в схемы аксиом и применений *modus ponens*, а структура рассуждения «спрятана» в номерах шагов.

Натуральный вывод

Натуральный вывод представляет собой систему, в которой доказательства строятся с использованием правил введения и устранения для каждой логической связки.

$$\frac{\Gamma, A \vdash B}{\Gamma \vdash A \rightarrow B} \rightarrow I \qquad \frac{\Gamma \vdash A \rightarrow B \quad \Gamma \vdash A}{\Gamma \vdash B} \rightarrow E$$

$$\frac{\Gamma \vdash A \quad \Gamma \vdash B}{\Gamma \vdash A \wedge B} \wedge I$$

$$\begin{array}{c}
\frac{\Gamma \vdash A \wedge B}{\Gamma \vdash A} \wedge E_1 \quad \frac{\Gamma \vdash A \wedge B}{\Gamma \vdash B} \wedge E_2 \\
\frac{\Gamma \vdash A}{\Gamma \vdash A \vee B} \vee I_1 \quad \frac{\Gamma \vdash B}{\Gamma \vdash A \vee B} \vee I_2 \\
\frac{\Gamma \vdash A \vee B \quad \Gamma, A \vdash C \quad \Gamma, B \vdash C}{\Gamma \vdash C} \vee E \\
\frac{\Gamma \vdash \perp}{\Gamma \vdash A} \perp E \quad \frac{}{\Gamma, A \vdash A} Ax
\end{array}$$

Пример. Формула $A \rightarrow (B \rightarrow A)$ выводится так:

$$\frac{\frac{\frac{}{A, B \vdash A} Ax}{A \vdash B \rightarrow A} \rightarrow I}{\vdash A \rightarrow (B \rightarrow A)} \rightarrow I$$

По сравнению с гильбертовой аксиоматикой натуральный вывод гораздо проще для восприятия человеком: правила введения и устранения связок напрямую отражают ход рассуждения. Вместе с тем такой формат хуже подходит для автоматической обработки: дерево доказательства с произвольным порядком введения гипотез и вложенными подвыводами не столь удобно разбирать алгоритмически, как линейные или почти линейные объекты.

Секвенциальное исчисление

Секвенциальное исчисление основано на понятии секвенции. Пусть $\mathcal{L}$ — множество формул пропозициональной логики, построенных из пропозициональных переменных, связок $\wedge, \vee, \rightarrow$ и константы $\perp$. Секвенцией называется пара (Γ, Δ) , записываемая

$$\Gamma \vdash \Delta,$$

где Γ (антецедент) и Δ (сукцедент) — конечные мультимножества формул из $\mathcal{L}$. Неформально секвенция $\Gamma \vdash \Delta$ читается как «из конъюнкции всех формул Γ выводима дизъюнкция формул Δ ». Секвенция с пустым антецедентом ($\vdash \Delta$) утверждает безусловную выводимость; секвенция с пустым сукцедентом ($\Gamma \vdash$) означает противоречивость Γ .

Ниже приведена система правил классического секвенциального исчисления высказываний (исчисление ЛК в стиле Генцена).

Аксиомы

$$\overline{A \vdash A}^{Ax} \quad \overline{\Gamma, \perp \vdash \Delta}^{\perp L}$$

Структурные правила

$$\begin{array}{cc} \frac{\Gamma \vdash \Delta}{\Gamma, A \vdash \Delta}^{LW} & \frac{\Gamma \vdash \Delta}{\Gamma \vdash \Delta, A}^{RW} \\ \frac{\Gamma, A, A \vdash \Delta}{\Gamma, A \vdash \Delta}^{LC} & \frac{\Gamma \vdash \Delta, A, A}{\Gamma \vdash \Delta, A}^{RC} \\ \frac{\Gamma, A, B, \Pi \vdash \Delta}{\Gamma, B, A, \Pi \vdash \Delta}^{LX} & \frac{\Gamma \vdash \Delta, A, B, \Pi}{\Gamma \vdash \Delta, B, A, \Pi}^{RX} \end{array}$$

Логические правила

$$\begin{array}{cc} \frac{\Gamma \vdash A, \Phi \quad \Pi, B \vdash \Psi}{\Gamma, \Pi, A \rightarrow B \vdash \Phi, \Psi} \rightarrow L & \frac{\Gamma, A \vdash B, \Delta}{\Gamma \vdash A \rightarrow B, \Delta} \rightarrow R \\ \frac{\Gamma \vdash A, \Delta \quad \Gamma \vdash B, \Delta}{\Gamma \vdash A \wedge B, \Delta} \wedge R \\ \frac{\Gamma, A \vdash \Delta}{\Gamma, A \wedge B \vdash \Delta} \wedge L_1 & \frac{\Gamma, B \vdash \Delta}{\Gamma, A \wedge B \vdash \Delta} \wedge L_2 \\ \frac{\Gamma, A \vdash \Delta \quad \Gamma, B \vdash \Delta}{\Gamma, A \vee B \vdash \Delta} \vee L \\ \frac{\Gamma \vdash A, \Delta}{\Gamma \vdash A \vee B, \Delta} \vee R_1 & \frac{\Gamma \vdash B, \Delta}{\Gamma \vdash A \vee B, \Delta} \vee R_2 \end{array}$$

Правило сечения

$$\frac{\Gamma \vdash A, \Delta \quad \Gamma, A \vdash \Sigma}{\Gamma \vdash \Delta, \Sigma}^{cut}$$

Секвенция $\vdash A \rightarrow A$ выводится без сечения и структурных правил:

$$\frac{\overline{A \vdash A}^{Ax}}{\vdash A \rightarrow A} \rightarrow R$$

Интуиционистская версия секвенциального исчисления получается ограничением на форму секвенций: интуиционистскими называются те секвенции, в которых справа находится не более одной формулы (то есть выводы строятся в подсистеме, где Δ либо пусто, либо состоит ровно из одной формулы). Ниже приведены правила интуиционистского секвенциального исчисления LJ.

Аксиомы

$$\overline{A \vdash A}^{Ax} \quad \overline{\Gamma, \perp \vdash \Delta}^{\perp L}$$

Структурные правила

$$\frac{\Gamma \vdash \Delta}{\Gamma, A \vdash \Delta}^{LW} \quad \frac{\Gamma, A, A \vdash \Delta}{\Gamma, A \vdash \Delta}^{LC} \quad \frac{\Gamma, A, B, \Gamma' \vdash \Delta}{\Gamma, B, A, \Gamma' \vdash \Delta}^{LX}$$

Логические правила

$$\frac{\Gamma \vdash A \quad \Gamma, B \vdash \Delta}{\Gamma, A \rightarrow B \vdash \Delta} \rightarrow L \quad \frac{\Gamma, A \vdash B}{\Gamma \vdash A \rightarrow B} \rightarrow R$$

$$\frac{\Gamma \vdash A \quad \Gamma \vdash B}{\Gamma \vdash A \wedge B} \wedge R$$

$$\frac{\Gamma, A \vdash \Delta}{\Gamma, A \wedge B \vdash \Delta} \wedge L_1 \quad \frac{\Gamma, B \vdash \Delta}{\Gamma, A \wedge B \vdash \Delta} \wedge L_2$$

$$\frac{\Gamma, A \vdash \Delta \quad \Gamma, B \vdash \Delta}{\Gamma, A \vee B \vdash \Delta} \vee L$$

$$\frac{\Gamma \vdash A}{\Gamma \vdash A \vee B} \vee R_1 \quad \frac{\Gamma \vdash B}{\Gamma \vdash A \vee B} \vee R_2$$

Правило сечения

$$\frac{\Gamma \vdash A \quad \Gamma, A \vdash \Delta}{\Gamma \vdash \Delta} cut$$

Ключевое отличие от ЛК — отсутствие правых структурных правил (ослабления, сокращения и перестановки справа), поскольку в сукцеденте находится не более одной формулы. Кроме того, в двухпосылочных правилах ($\rightarrow R$, cut) сукцеденты посылок не объединяются: одна из посылок обязана вывести ровно одну «вспомогательную» формулу, а другая — передать результат в единственную формулу заключения.

Секвенциальный формат удобен для реализации на ЭВМ: правила локальны, вывод представляется деревом с предсказуемой структурой, что облегчает поиск вывода и проверку корректности. Именно на секвенциальных исчислениях и их модификациях (ориентированные варианты, ограничения на правила, стратегии устранения сечения и т. д.) строятся многие современные автоматические средства; они служат основой для развитых систем в духе LJT , LJT_{SAT} и аналогов.

1.3 Типизированное λ -исчисление

В теории доказательств основной задачей является поиск доказательства: его структура, сравнение различных выводов одной и той же формулы. Для конструктивной (интуиционистской) логики это особенно существенно, поскольку доказательство трактуется как конструкция в смысле интерпретации интуиционистской логики.

Удобно явно указывать терм (объект доказательства) рядом с формулой: запись $M : \psi$ означает, что M — доказательство ψ , при контексте гипотез Γ используют суждение $\Gamma \vdash M : \psi$.

Правило устранения импликации в натуральном выводе с аннотациями: если $\Gamma \vdash M : A \rightarrow B$ и $\Gamma \vdash N : A$, то доказательством B задаётся применение $M N$:

$$\frac{\Gamma \vdash M : A \rightarrow B \quad \Gamma \vdash N : A}{\Gamma \vdash M N : B} \rightarrow E$$

Аксиома гипотезы: переменная, помеченная формулой A , служит доказательством A в расширенном контексте:

$$\frac{}{\Gamma, x : A \vdash x : A} Ax$$

Введение импликации: доказательство $A \rightarrow B$ из подвывода с гипотезой $x : A$ записывают как λ -абстракцию по x :

$$\frac{\Gamma, x : A \vdash M : B}{\Gamma \vdash \lambda x. M : A \rightarrow B} \rightarrow I$$

Таким образом аннотированный натуральный вывод совпадает по виду с правилами типизации в λ -исчислении. В интерпретации интуиционистской логики конструкция для импликации $A \rightarrow B$ — это функция, сопоставляющая конструкции для A конструкции для B .

Аннотирование правил LJ (импликативный фрагмент)

Подход с аннотированным термами к формулам можно распространить и на секвенциальное исчисление:

$$\frac{}{\Gamma, x : A \vdash x : A} Ax \quad \frac{\Gamma, x : A \vdash M : B}{\Gamma \vdash \lambda x. M : A \rightarrow B} \rightarrow R$$

$$\frac{\Gamma \vdash M : A \quad \Delta, z : B \vdash N : C}{\Gamma, \Delta, u : A \rightarrow B \vdash N[z := u M] : C} \rightarrow L$$

Основная суть аннотирования в том, чтобы устранять свободные переменные у терма справа. Под свободными переменными понимаются только те свободные относительно терма справа, которых нет слева. Проще всего объяснить данную идею на правиле $(\rightarrow L)$. В нем мы в результате справа получаем C , но среди посылок у нас пропадает B . Из этого следует, что нужно заменить все вхождения терма z в терме N , на те, что остались слева. Также стоит обосновать почему мы можем использовать терм M при замене. Тут мы знаем, что все свободные переменные терма M содержатся в Γ (левая посылка $\Gamma \vdash M : A$), поэтому подстановка не вносит новых свободных переменных.

В правиле $(\rightarrow R)$ мы убираем из левой части $x : A$, поэтому в правой должны как то убрать свободную переменную x в терме M . Для этого мы просто делаем ее связанной при помощи λ -абстракции.

Суперскрипты для произведения и суммы

Аннотировать можно также и остальные правила секвенциального исчисления, но для этого требуется ввести лямбда-термы для произведения и суммы типов. Для произведения подойдет терм пары следующего вида:

$$\langle M, N \rangle = \lambda x. x M N$$

$$\pi_1 = \lambda p. p(\lambda x y. x)$$

$$\pi_2 = \lambda p. p(\lambda x y. y)$$

Здесь π_1 и π_2 — проекции на первую и вторую компоненту пары. Причем $\pi_1 \langle M, N \rangle \rightarrow_\beta M$ и $\pi_2 \langle M, N \rangle \rightarrow_\beta N$.

Для суммы типов можно использовать следующие термы:

$$L = \lambda e c_1 c_2. c_1 e$$

$$R = \lambda e c_1 c_2. c_2 e$$

$$case\ M\ of\ [x]P\ or\ [y]Q = M(\lambda x. P)(\lambda y. Q)$$

Тут L и R — конструкторы суммы, а $case$ — элиминатор. В выражении $case$ M может быть вида LV либо RV , где V какой либо терм. Таким образом

$$\begin{aligned} case\ LM\ of\ [x]P\ or\ [y]Q &\rightarrow_{\beta} P[x := M] \\ case\ RM\ of\ [x]P\ or\ [y]Q &\rightarrow_{\beta} Q[y := M] \end{aligned}$$

Аннотирование правил LJ (полный фрагмент)

Теперь благодаря введенным термам для произведения и суммы типов, можно аннотировать остальные правила секвенциального исчисления. Для простоты будем аннотировать только логические правила, так как в структурные аннотируются естественно.

$$\begin{aligned} &\frac{\Gamma \vdash a : A \quad \Gamma \vdash b : B}{\Gamma \vdash \langle a, b \rangle : A \wedge B} \wedge_R \\ &\frac{\Gamma, a : A \vdash d : C}{\Gamma, p : A \wedge B \vdash d[a := \pi_1 p] : C} \wedge_{L_1} \quad \frac{\Gamma, b : B \vdash d : C}{\Gamma, p : A \wedge B \vdash d[b := \pi_2 p] : C} \wedge_{L_2} \\ &\frac{\Gamma, a : A \vdash d_1 : C \quad \Gamma, b : B \vdash d_2 : C}{\Gamma, p : A \vee B \vdash case\ p\ of\ [a]d_1\ or\ [b]d_2 : C} \vee_L \\ &\frac{\Gamma \vdash a : A}{\Gamma \vdash La : A \vee B} \vee_{R_1} \quad \frac{\Gamma \vdash b : B}{\Gamma \vdash Rb : A \vee B} \vee_{R_2} \end{aligned}$$

2 Конструкторская часть

2.1 Исчисление LJТ

Исчисление LJ в формулировке Дайкхоффа

Секвенция — пара $\Gamma \vdash G$, где Γ — мультимножество формул (антецедент), G — формула (сукцедент). Поскольку антецедент является мультимножеством, перестановка формул допускается автоматически, и структурное правило перестановки не требуется. Формулы строятся из пропозициональных переменных, связок $\wedge, \vee, \rightarrow$ и константы $\perp$. Константа $\perp$ не считается атомом.

Правила исчисления LJ (в формулировке, следующей Дайкхоффу) приведены ниже. Выделенная формула в заключении каждого правила называется главной (principal).

Аксиома и абсурд

$$\frac{}{\underline{A}, \Gamma \vdash A} \text{Ax}, A \text{ — атом} \qquad \frac{}{\underline{\perp}, \Gamma \vdash G} \perp L$$

Конъюнкция

$$\frac{A, B, \Gamma \vdash G}{\underline{A \wedge B}, \Gamma \vdash G} \wedge L \qquad \frac{\Gamma \vdash A \quad \Gamma \vdash B}{\Gamma \vdash \underline{A \wedge B}} \wedge R$$

Дизъюнкция

$$\frac{A, \Gamma \vdash G \quad B, \Gamma \vdash G}{\underline{A \vee B}, \Gamma \vdash G} \vee L \qquad \frac{\Gamma \vdash A_i}{\Gamma \vdash \underline{A_1 \vee A_2}} \vee R$$

Импликация

$$\frac{A \rightarrow B, \Gamma \vdash A \quad B, \Gamma \vdash G}{\underline{A \rightarrow B}, \Gamma \vdash G} \rightarrow L \qquad \frac{A, \Gamma \vdash B}{\Gamma \vdash \underline{A \rightarrow B}} \rightarrow R$$

Ключевая особенность данной формулировки — правило $(\rightarrow L)$: главная формула $A \rightarrow B$ сохраняется в левой посылке. Тем самым правило сокращения встроено в $(\rightarrow L)$, и ни сокращение, ни ослабление, ни перестановка не требуются как примитивные правила: все три допустимы.

Проблема завершаемости

Для формализации завершаемости вводится вес формулы $\text{wt}(A)$:

$$\begin{aligned}\text{wt}(p) &= \text{wt}(\perp) = 1, \\ \text{wt}(A \wedge B) &= \text{wt}(A) + \text{wt}(B) + 2, \\ \text{wt}(A \vee B) &= \text{wt}(A) + \text{wt}(B) + 1, \\ \text{wt}(A \rightarrow B) &= \text{wt}(A) + \text{wt}(B) + 1.\end{aligned}$$

Формулы антецедента и сукцедент секвенции образуют мультимножество; на мультимножествах определяется отношение $\gg$ (мультимножественное упорядочение Дершовица–Манна), которое фундировано: если из мультимножества удалить один или несколько элементов и заменить их произвольным числом элементов меньшего веса, результат строго меньше исходного.

Во всех правилах LJ, кроме $(\rightarrow L)$, каждая посылка строго меньше заключения в смысле $\gg$: главная формула заменяется подформулами меньшего веса. Однако в правиле $(\rightarrow L)$ левая посылка $A \rightarrow B, \Gamma \vdash A$ содержит ту же формулу $A \rightarrow B$, что и заключение, поэтому $\gg$ -убывание не гарантировано. Как следствие, обратный поиск вывода от корня к листьям (root-first proof search) может заиклиться: применение $(\rightarrow L)$ к секвенции $(D_1 \rightarrow D_2) \rightarrow C, \Gamma \vdash G$ порождает подцель $(D_1 \rightarrow D_2) \rightarrow C, \Gamma \vdash D_1 \rightarrow D_2$, в которой главная формула по-прежнему присутствует и может быть использована повторно.

Правила исчисления LJТ

В 1992 году Дайкхофф предложил исчисление LJТ (также известное как G4ip), в котором правило $(\rightarrow L)$ заменяется четырьмя правилами, определяемыми структурой левой части (антецедента) главной импликации $D \rightarrow C$. Все остальные правила сохраняются без изменений.

Случай 1. $D = p$ — пропозициональная переменная:

$$\frac{B, p, \Gamma \vdash G}{p \rightarrow B, p, \Gamma \vdash G} \rightarrow_{L_1}$$

Случай 2. $D = C_1 \wedge C_2$:

$$\frac{C_1 \rightarrow (C_2 \rightarrow B), \Gamma \vdash G}{(C_1 \wedge C_2) \rightarrow B, \Gamma \vdash G} \rightarrow_{L_2}$$

Случай 3. $D = C_1 \vee C_2$:

$$\frac{C_1 \rightarrow B, C_2 \rightarrow B, \Gamma \vdash G}{(C_1 \vee C_2) \rightarrow B, \Gamma \vdash G} \rightarrow_{L_3}$$

Случай 4. $D = C_1 \rightarrow C_2$:

$$\frac{C_2 \rightarrow B, \Gamma \vdash C_1 \rightarrow C_2 \quad B, \Gamma \vdash G}{(C_1 \rightarrow C_2) \rightarrow B, \Gamma \vdash G} \rightarrow_{L_4}$$

Во всех четырёх правилах главная формула не сохраняется ни в одной из посылок: она заменяется формулами строго меньшего веса. Действительно:

- $(\rightarrow L_1)$: формула $p \rightarrow B$ ($\text{wt} = \text{wt}(B) + 2$) заменяется на B ;
- $(\rightarrow L_2)$: $\text{wt}((C_1 \wedge C_2) \rightarrow B) = \text{wt}(C_1) + \text{wt}(C_2) + \text{wt}(B) + 3$, а $\text{wt}(C_1 \rightarrow (C_2 \rightarrow B)) = \text{wt}(C_1) + \text{wt}(C_2) + \text{wt}(B) + 2$;
- $(\rightarrow L_3)$: одна формула веса $\text{wt}(C_1) + \text{wt}(C_2) + \text{wt}(B) + 2$ заменяется двумя формулами весов $\text{wt}(C_1) + \text{wt}(B) + 1$ и $\text{wt}(C_2) + \text{wt}(B) + 1$, каждая строго меньше;
- $(\rightarrow L_4)$: формула $(C_1 \rightarrow C_2) \rightarrow B$ заменяется на $C_2 \rightarrow B$ (в левой посылке) и B (в правой), обе строго меньше.

Таким образом, каждая посылка каждого правила LJТ строго меньше заключения в мультимножественном упорядочении $\gg$, что гарантирует завершаемость обратного поиска вывода. Правила ослабления и сокращения допустимы, но не входят в систему как примитивные.

Обратный поиск вывода

Обратный поиск вывода (root-first proof search) в LJТ — процедура, которая по данной секвенции $\Gamma \vdash G$ определяет, выводима ли она. Процедура сопоставляет секвенцию с заключением одного из правил и рекурсивно доказывает посылки. Все правила LJТ, кроме $(\vee R)$ и $(\rightarrow L_4)$, обратимы: из доказуемости заключения следует доказуемость каждой посылки. При применении обратимого правила возврат (backtracking) не требуется: если хотя бы одна посылка недоказуема, заключение также недоказуемо. Правило $(\rightarrow L_4)$ является правообратимым: из доказуемости заключения следует доказуемость правой посылки $\Gamma, C \vdash G$, тогда как левая посылка может быть недоказуема. Поэтому $(\rightarrow L_4)$ может рассматриваться как правило с одной посылкой и побочным условием (side-condition), а вызов для правой посылки является хвостовой рекурсией.

Возврат необходим только при применении $(\vee R)$ (выбор дизъюнкта) и $(\rightarrow L_4)$ (выбор импликационной клаузы из антецедента).

Псевдокод процедуры приведён в Приложении А (листинг 1). Поскольку множество формул в каждой посылке строго меньше в мультимножественном упорядочении $\gg$, рекурсия завершается.

Пример вывода

Покажем вывод формулы $((a \vee (a \rightarrow b)) \rightarrow b) \rightarrow b$ в системе LJТ с λ -термовыми аннотациями.

$$\frac{\frac{\frac{}{g_1 : a \rightarrow b, h : b \rightarrow b \vdash g_1 : a \rightarrow b} Ax \quad \frac{}{y : b, g_1 : a \rightarrow b \vdash y : b} Ax}{\frac{}{g_1 : a \rightarrow b, g_2 : (a \rightarrow b) \rightarrow b \vdash g_2 g_1 : b} \rightarrow L_4} \rightarrow L_3}{\frac{}{f : (a \vee (a \rightarrow b)) \rightarrow b \vdash f(R(\lambda x. f(Lx))) : b} \rightarrow R} \vdash \lambda f. f(R(\lambda x. f(Lx))) : ((a \vee (a \rightarrow b)) \rightarrow b) \rightarrow b$$

Более сложный пример — вывод формулы $(((((a \wedge b) \rightarrow c) \rightarrow ((a \rightarrow c) \vee (b \rightarrow c))) \rightarrow c) \rightarrow c$ с аннотированием термами — приведён в приложении Б (с. 34).

2.2 Исчисление LJT_{SAT}

Исчисление LJТ предоставляет завершающуюся процедуру поиска вывода, однако не даёт свидетельства недоказуемости: при неудаче поиска остаётся лишь факт отсутствия вывода, но не контрмодель. Клэссен и Розен [1] предложили инструмент *intuit* — доказатель для интуиционистской пропозициональной логики, построенный поверх инкрементального SAT-решателя по схеме SMT (Satisfiability Modulo Theories). Фиорентини, Горе и Грэм-Ленгран [2] дали теоретико-доказательственное обоснование этого подхода, сформулировав исчисление LJT_{SAT} , которое объединяет поиск вывода в LJТ [3] с использованием SAT-решателя и реализует полную процедуру принятия решения с двумя исходами: вывод в LJT_{SAT} либо контрмодель Крипке.

Клаузальная форма

Исчисление LJT_{SAT} работает с секвенциями в клаузальной форме $R, X \Rightarrow q$, где:

- R — множество плоских клауз вида $\bigwedge A_1 \rightarrow \bigvee A_2$, где $A_1, A_2 \subseteq Atm$
- конечные множества атомов;
- X — множество импликационных клауз вида $(a \rightarrow b) \rightarrow c$, где a, b, c — атомы;
- q — атом (целевая формула).

Произвольная формула α интуиционистской логики преобразуется в клаузальную форму процедурой клаузации [1].

Шаг 1. Формула α приводится к виду $B \rightarrow q$, где q — атом. Если α не имеет такого вида, вводится свежий атом q и формула заменяется на $(\alpha \rightarrow q) \rightarrow q$ (эквивалентность в интуиционистской логике).

Шаг 2. Формула B раскладывается в множества R и X с помощью правил перезаписи вида $A \rightsquigarrow B_1, \dots, B_n$, заменяющих допущение A на эквивалентный набор допущений B_i . Если формула ещё не является импликацией, применяется правило

$$A \rightsquigarrow \top \rightarrow A.$$

Далее применяются структурные правила, упрощающие вид импликации:

$$\begin{aligned} (A \vee B) \rightarrow a &\rightsquigarrow A \rightarrow a, \quad B \rightarrow a, \\ a \rightarrow (A \wedge B) &\rightsquigarrow a \rightarrow A, \quad a \rightarrow B, \\ A \rightarrow (B \rightarrow C) &\rightsquigarrow (A \wedge B) \rightarrow C, \end{aligned}$$

где a — атом или константа $\perp$, $\top$. Для введения атомов на нужных позициях используются правила с введением свежих переменных (обозначены строчными буквами справа):

$$\begin{aligned} A \rightarrow (\dots \vee B \vee \dots) &\rightsquigarrow A \rightarrow (\dots \vee b \vee \dots), \quad b \rightarrow B, \\ (\dots \wedge A \wedge \dots) \rightarrow B &\rightsquigarrow (\dots \wedge a \wedge \dots) \rightarrow B, \quad A \rightarrow a, \\ (A \rightarrow B) \rightarrow C &\rightsquigarrow (a \rightarrow B) \rightarrow C, \quad A \rightarrow a, \\ (A \rightarrow B) \rightarrow C &\rightsquigarrow (A \rightarrow b) \rightarrow C, \quad B \rightarrow b, \\ (A \rightarrow B) \rightarrow C &\rightsquigarrow (A \rightarrow B) \rightarrow c, \quad c \rightarrow C. \end{aligned}$$

Здесь a, b, c — свежие атомы, не встречающиеся в формуле. Каждая подформула именуется не более одного раза, поэтому суммарный размер (R, X, q) линеен относительно размера α .

Результатом клаузификации является тройка $clausal(\alpha) = (R, X, q)$, для которой выполняется $\vdash_{ipl} \alpha$ тогда и только тогда, когда $R, X \vdash_{ipl} q$.

В клаузальной форме из правил LJТ применимо только правило $(\rightarrow L_4)$, так как антецеденты импликационных клауз сами являются импликациями. Плоские клаузы не содержат вложенных связок и допускают классическое рассуждение: $R \vdash_{cpl} q$ тогда и только тогда, когда $R \vdash_{ipl} q$, что позволяет делегировать работу с ними SAT-решателю.

Правила исчисления LJT_{SAT}

Исчисление LJT_{SAT} состоит из трёх правил.

Правило (cpl). Если множество плоских клауз R классически влечёт q (что проверяется SAT-решателем), секвенция выводима:

$$\frac{R \vdash_{cpl} q}{R, X \Rightarrow q} \text{ } cpl$$

Правило (cut_{ipl}). Интуиционистское сечение: если левая посылка является интуиционистской теоремой и плоская клауза φ позволяет вывести правую посылку, то заключение выводимо:

$$\frac{R_1, X_1 \vdash_{ipl} \varphi \quad \varphi, R_2, X_2 \Rightarrow q}{R_1, R_2, X_1, X_2 \Rightarrow q} \text{ } cut_{ipl}$$

Правило (ljt). Обобщённое правило левого введения импликации, соответствующее $(\rightarrow L_4)$ из LJТ, адаптированное к клаузальной форме. Пусть $\iota = (a \rightarrow b) \rightarrow c \in X$, $X_\iota = X \setminus \{\iota\}$ и A_1 — множество атомов. Тогда:

$$\frac{R_1, b \rightarrow c, X_\iota, A_1 \Rightarrow b \quad R_2, \bigwedge(A_1 \setminus \{a\}) \rightarrow c, X, \iota \Rightarrow q}{R_1, R_2, X, \iota \Rightarrow q} \text{ } ljт$$

Правило (ljt) обобщает $(\rightarrow L_4)$ из LJТ в двух направлениях. Во-первых, допускается расщепление контекста (R_1 и R_2 могут различаться). Во-вторых, в левую посылку могут быть добавлены дополнительные атомарные допущения A_1 , что расширяет область применимости правила; ценой этого расширения является ослабление выученной клаузы в правой посылке: вместо атома c появляется плоская клауза $\bigwedge(A_1 \setminus \{a\}) \rightarrow c$.

При $R_1 = R_2 = R$ и $A_1 = \{a\}$ (то есть $\bigwedge(A_1 \setminus \{a\}) \rightarrow c$ вырождается в атом c) правило (ljt) сводится к $(\rightarrow L_4)$ исчисления LJТ, поскольку наличие c в правой посылке делает импликационную клаузу ι избыточной.

Процедура поиска вывода

Процедура поиска вывода в LJT_{SAT} состоит из двух функций: главной $LJTSatMain$ и рекурсивной $LJTSat$ (псевдокод приведён в Приложении В, листинги 2 и 3). Обе функции используют единый инкрементальный SAT-решатель s : клаузы могут добавляться в s , но не удаляться; запросы принимают переменное множество атомарных допущений.

Функция $LJTSatMain(R_0, X_0, q_0)$ инициализирует SAT-решатель s и загружает в него все плоские клаузы из R_0 , а также для каждой импликационной клаузы $(a \rightarrow b) \rightarrow c \in X_0$ — клаузу $b \rightarrow c$ (следствие исходной задачи). После этого управление передаётся функции $LJTSat$.

Функция $LJTSat(R, X, A, q)$ реализует цикл CEGAR (Counter-Example Guided Abstraction Refinement). Она запрашивает SAT-решатель: выполняется ли $R(s), A \vdash_{cpl} q$? Если да, секвенция выводима по правилу (cpl) ; SAT-решатель возвращает подмножество $A' \subseteq A$, достаточное для вывода. В противном случае SAT-решатель возвращает классическую контрмодель M — множество атомов, удовлетворяющее $R(s) \cup A$, но не q .

Контрмодель M проверяется на совместимость с импликационными клаузами. Для каждой $\iota = (a \rightarrow b) \rightarrow c \in X$, такой что $a \notin M, b \notin M$ и $c \notin M$ (то есть M не удовлетворяет ι тривиально), выполняется рекурсивный вызов для левой посылки правила (ljt) : $LJTSat(R \cup \{b \rightarrow c\}, X_\iota, M \cup \{a\}, b)$, где $X_\iota = X \setminus \{\iota\}$. Если вызов вернул контрмодель K_1 , она сохраняется для последующей склейки. Если же вызов успешен и возвращает множество атомов A_1 , из него формируется выученная клауза $\tilde{\varphi} = \bigwedge(A_1 \setminus \{a\}) \rightarrow c$, которая добавляется в SAT-решатель. Затем $LJTSat$ вызывается рекурсивно для правой посылки: $LJTSat(R \cup \{\tilde{\varphi}\}, X, A, q)$. Этот вызов является хвостовой рекурсией (правообратимость правила (ljt)) и соответствует перезапуску цикла CEGAR с уточнённым множеством клауз.

Если для всех применимых импликационных клауз рекурсивные вызовы вернули контрмодели, формула недоказуема, и из M и накопленных подмоделей конструируется контрмодель Крипке методом склейки.

Завершаемость процедуры гарантируется убыванием пары $(X, \mathcal{M}(R))$ по лексикографическому порядку, где $\mathcal{M}(R)$ — множество классических моделей, удовлетворяющих R . Каждая выученная клауза $\tilde{\varphi}$ исключает хотя бы одну модель (а именно M) из $\mathcal{M}(R)$, что гарантирует строгое убывание по второй компоненте при неизменной первой.

Построение контрмодели Крипке

При неудаче поиска вывода процедура строит контрмодель Крипке методом склейки (gluing). Каждый неудачный лист дерева поиска задаёт мир контрмодели, а рекурсивная структура вызовов определяет отношение достижимости.

Пусть $\Theta = (K_\iota)_{\iota \in X_M}$ — семейство крипке-моделей, построенных рекурсивными вызовами для импликационных клауз $X_M = \{\iota \in X \mid a \notin M, b \notin M, c \notin M\}$, и M — классическая контрмодель. Тогда модель $\text{Mod}(r, \Theta, M) = \langle W, \preceq, r, V \rangle$ определяется так:

- $W = \{r\} \cup \bigcup_{\iota \in X_M} W_\iota$, где W_ι — множество миров модели K_ι ;
- r — новый корневой мир с оценкой $V(r) = M$;
- $r \preceq r_\iota$ для каждого корня r_ι модели K_ι ;
- $\preceq$ — рефлексивно-транзитивное замыкание отношений достижимости всех подмоделей и связей $r \preceq r_\iota$.

Если $\Theta = \emptyset$ (все импликационные клаузы имеют хотя бы один компонент, истинный в M), контрмодель состоит из единственного мира r с $V(r) = M$. Условие наследуемости выполняется по построению: корни подмоделей r_ι удовлетворяют $r_\iota \models M$, поскольку $M \subseteq V(r_\iota)$.

Корректность построения гарантируется тем, что в корне r вынуждены все формулы из $R \cup X$, но не вынуждена целевая формула q , что делает построенную структуру контрмоделью для исходной секвенции.

2.3 Аннотирование термами

В исследовательской части было показано, что правила секвенциального исчисления LJ могут быть аннотированы λ -термами в соответствии с изоморфизмом Карри–Говарда: каждому выводу сопоставляется терм, типом которого является выведенная формула. Тот же подход применим к исчислениям LJТ и к правилам

клаузификации, что позволяет по найденному выводу извлечь программу (свидетель населённости типа).

Аннотирование правил LJТ

Правила LJТ, общие с LJ (аксиома, $\perp L$, $\wedge L$, $\wedge R$, $\vee L$, $\vee R$, $\rightarrow R$), аннотируются так же, как в LJ. Специфика LJТ состоит в четырёх правилах левого введения импликации.

Правило ($\rightarrow L_1$). Атом p присутствует в антецеденте, поэтому терм для B получается применением f к x :

$$\frac{\Gamma, x : p, y : B \vdash t : G}{\Gamma, x : p, f : p \rightarrow B \vdash t[y := f x] : G} \rightarrow L_1$$

Правило ($\rightarrow L_2$). Импликация $(A \wedge B) \rightarrow C$ заменяется на $A \rightarrow (B \rightarrow C)$:

$$\frac{\Gamma, g : A \rightarrow (B \rightarrow C) \vdash t : G}{\Gamma, f : (A \wedge B) \rightarrow C \vdash t[g := \lambda x. \lambda y. f \langle x, y \rangle] : G} \rightarrow L_2$$

Правило ($\rightarrow L_3$). Импликация $(A \vee B) \rightarrow C$ заменяется парой $A \rightarrow C$ и $B \rightarrow C$:

$$\frac{\Gamma, g_1 : A \rightarrow C, g_2 : B \rightarrow C \vdash t : G}{\Gamma, f : (A \vee B) \rightarrow C \vdash t[g_1 := \lambda x. f(Lx), g_2 := \lambda y. f(Ry)] : G} \rightarrow L_3$$

Правило ($\rightarrow L_4$). Единственное правило с двумя посылками. В левой посылке формула $(C_1 \rightarrow C_2) \rightarrow B$ заменяется на $C_2 \rightarrow B$:

$$\frac{\Gamma, g : C_2 \rightarrow B \vdash s : C_1 \rightarrow C_2 \quad \Gamma, y : B \vdash t : G}{\Gamma, f : (C_1 \rightarrow C_2) \rightarrow B \vdash t[y := f s', g := \lambda z. f(Kz)] : G} \rightarrow L_4$$

где $K = \lambda a. \lambda b. a$ — комбинатор константы, а $s' = s[g := \lambda z. f(Kz)]$. Интуиция подстановки: имея $f : (C_1 \rightarrow C_2) \rightarrow B$ и значение $z : C_2$, можно построить константную функцию $Kz = \lambda x^{C_1}. z : C_1 \rightarrow C_2$ и применить f , получив $f(Kz) : B$. Таким образом $g := \lambda z. f(Kz) : C_2 \rightarrow B$. Развёрнутый пример подстановки приведён в приложении Б (с. 34).

Клаузификация как секвенциальные правила

Каждое правило перезаписи, используемое при клаузификации формулы в клаузальную форму $R, X \Rightarrow q$, может быть представлено как допустимое правило секвенциального исчисления. Это позволяет аннотировать клаузификацию

λ -термами: каждому шагу перезаписи соответствует терм, преобразующий доказательство в клаузуальной форме в доказательство исходной формулы.

Рассмотрим правила клаузификации. Каждое из них имеет вид: секвенция с допущением $B_1, \dots, B_n$ выводима тогда и только тогда, когда выводима секвенция с допущением A . Аннотирование состоит в указании терма, реализующего это преобразование.

Начальное правило. Формула, не являющаяся импликацией, оборачивается:

$$\frac{\Gamma, f : \top \rightarrow A \vdash t : G}{\Gamma, x : A \vdash t[f := \lambda u. x] : G}$$

Дизъюнкция слева от стрелки:

$$\frac{\Gamma, f_1 : A \rightarrow a, f_2 : B \rightarrow a \vdash t : G}{\Gamma, f : (A \vee B) \rightarrow a \vdash t[f_1 := \lambda x. f(Lx), f_2 := \lambda y. f(Ry)] : G}$$

Конъюнкция справа от стрелки:

$$\frac{\Gamma, f_1 : a \rightarrow A, f_2 : a \rightarrow B \vdash t : G}{\Gamma, f : a \rightarrow (A \wedge B) \vdash t[f_1 := \lambda x. \pi_1(fx), f_2 := \lambda x. \pi_2(fx)] : G}$$

Каррирование:

$$\frac{\Gamma, g : (A \wedge B) \rightarrow C \vdash t : G}{\Gamma, f : A \rightarrow (B \rightarrow C) \vdash t[g := \lambda p. f(\pi_1 p)(\pi_2 p)] : G}$$

Правила именования подформул. При клаузификации для сложной подформулы B вводится свежий атом b , играющий роль определяющего сокращения: $b \equiv B$. Поскольку b и B отождествляются на уровне типов, все правила именования имеют тривиальную термовую аннотацию. Рассмотрим два характерных случая.

Именованье в положительной позиции (например, в сукцеденте дизъюнкции). Подформула B заменяется на b , добавляется клауза $b \rightarrow B$:

$$\frac{\Gamma, f' : A \rightarrow (\dots \vee b \vee \dots), h : b \rightarrow B \vdash t : G}{\Gamma, f : A \rightarrow (\dots \vee B \vee \dots) \vdash t[f' := f, h := \lambda x. x] : G}$$

Именованье в отрицательной позиции (например, в антецеденте дизъюнкции). Подформула B заменяется на b , добавляется клауза $B \rightarrow b$:

$$\frac{\Gamma, f' : (\dots \vee b \vee \dots) \rightarrow a, h : B \rightarrow b \vdash t : G}{\Gamma, f : (\dots \vee B \vee \dots) \rightarrow a \vdash t[f' := f, h := \lambda x. x] : G}$$

В обоих случаях подстановка тождественна: $f' := f$ и $h := \lambda x. x$, поскольку b является определительным сокращением для B и значения этих типов совпадают. Аналогичная аннотация применяется ко всем остальным позициям именованя (конъюнкция, импликация).

Совокупность аннотированных правил клаузификации и аннотированных правил LJТ позволяет по выводу в LJT_{SAT} построить λ -терм, являющийся свидетелем населённости исходного типа.

Извлечение терма из клаузальной формы

При клаузификации формула α приводится к виду $(\alpha \rightarrow q) \rightarrow q$, где q — свежий атом, не входящий в α . Между α и $(\alpha \rightarrow q) \rightarrow q$ имеет место эквивалентность доказуемости: $\vdash_{ipl} \alpha$ тогда и только тогда, когда $\vdash_{ipl} (\alpha \rightarrow q) \rightarrow q$.

Допустим, процедура поиска вывода в LJT_{SAT} завершилась успешно и по аннотированному выводу построен замкнутый λ -терм $M : (\alpha \rightarrow q) \rightarrow q$. Поскольку q — свежий атом, не входящий в α , к доказательству применима равномерная подстановка: замена атома q на произвольную формулу ψ во всём выводе сохраняет его корректность. При этом структура λ -терма M не изменяется — подстановка затрагивает только типовые аннотации, но не сам терм.

Выполним подстановку $q := \alpha$. Терм M приобретает тип $(\alpha \rightarrow \alpha) \rightarrow \alpha$. Тождественная функция $I = \lambda x. x$ имеет тип $\alpha \rightarrow \alpha$, поэтому применение

$$M I \rightarrow_{\beta} N$$

даёт терм $N : \alpha$, являющийся свидетелем населённости типа α .

2.4 Исчисление $C^{\rightarrow}$

Фиорентини [4] предложил упрощённый вариант исчисления LJT_{SAT} , названный $C^{\rightarrow}$, а также процедуру поиска вывода `proveR` (prove with Restart), реализованную в инструменте `intuitR`. По сравнению с LJT_{SAT} исчисление $C^{\rightarrow}$ не содержит правила сечения и правила (*ljt*); вместо них используется единственное правило (*cpl*₁), в котором левая посылка проверяется классически. Благодаря

этому выводы в $C^{\rightarrow}$ имеют линейную структуру (единственная ветвь), что существенно упрощает как сам вывод, так и извлечение из него λ -терма.

Правила исчисления

Исчисление $C^{\rightarrow}$ работает с r -секвенциями $R, X \Rightarrow g$ (того же вида, что и в LJT_{SAT}) и состоит из двух правил.

Правило (cpl_0). Аксиома: если плоские клаузы R классически влекут g , секвенция выводима:

$$\frac{R \vdash_{cpl} g}{R, X \Rightarrow g} \text{ } cpl_0$$

Правило (cpl_1). Пусть $(a \rightarrow b) \rightarrow c \in X$ и $A \subseteq \mathcal{V}$ — множество пропозициональных переменных (локальные допущения). Если $R, A \vdash_{cpl} b$, формируется выученная клауза $\varphi = \bigwedge(A \setminus \{a\}) \rightarrow c$:

$$\frac{R, A \vdash_{cpl} b \quad R \cup \{\varphi\}, X \Rightarrow g}{R, X \Rightarrow g} \text{ } cpl_1$$

Вывод в $C^{\rightarrow}$ представляет собой линейную последовательность применений правила (cpl_1), завершающуюся применением (cpl_0): на каждом шаге k множество плоских клауз R_k пополняется выученной клаузой φ_k , и следующий шаг работает с $R_{k+1} = R_k \cup \{\varphi_k\}$.

Корректность правила (cpl_1) основана на следующем свойстве: если $R, A \vdash_{cpl} b$, то $R, (a \rightarrow b) \rightarrow c \vdash_{ipl} \varphi$. Действительно, из $R, A \vdash_{cpl} b$ по лемме о плоских клаузах следует $R, A \vdash_{ipl} b$, откуда $R, A \setminus \{a\} \vdash_{ipl} a \rightarrow b$, и с учётом импликационной клаузы получаем $R, (a \rightarrow b) \rightarrow c, A \setminus \{a\} \vdash_{ipl} c$, что даёт $R, (a \rightarrow b) \rightarrow c \vdash_{ipl} \varphi$.

Аннотирование правил

Как и для LJT , r -секвенции $R, X \Rightarrow g$ могут быть аннотированы λ -термами: каждой плоской клаузе из R сопоставляется переменная, импликационной клаузе $(a \rightarrow b) \rightarrow c \in X$ — переменная $h : (a \rightarrow b) \rightarrow c$, заключению — терм $t : g$.

Правило (cpl_0). Если $R \vdash_{cpl} g$, терм $t : g$ строится из переменных плоских клауз R ; множество X в аннотации не участвует:

$$\frac{R \vdash_{cpl} g}{R, X \vdash t : g}^{cpl_0}$$

Сечение (cut_{ipl}). Правило (cpl_1) реализует сечение с синтезированным доказательством выученной клаузы. Общая схема:

$$\frac{\Gamma \vdash s : \varphi \quad \Delta, f : \varphi \vdash t : g}{\Gamma, \Delta \vdash t[f := s] : g}^{cut_{ipl}}$$

Правило (cpl_1). Пусть $\iota = (a \rightarrow b) \rightarrow c \in X$, $A \subseteq \mathcal{V}$ и $R, A \vdash_{cpl} b$. SAT-решатель возвращает минимальное подмножество $A' \subseteq A$, достаточное для вывода; по лемме о плоских клаузах существует терм $p : b$ в контексте R и переменных $x_q : q$ для $q \in A'$. Выученная клауза $\varphi = \bigwedge(A \setminus \{a\}) \rightarrow c$ в каррированном виде имеет тип $u_1 \rightarrow (\dots \rightarrow c)$ для атомов $u_i \in A \setminus \{a\}$. Терм для φ :

$$s = \lambda \vec{u}. h(\lambda x^a. p'),$$

где $\vec{u}$ — последовательность абстракций по атомам из $A \setminus \{a\}$, а p' — терм p с подстановкой соответствующих $\vec{u}$ вместо переменных из A' . Аннотированное правило:

$$\frac{R, A \vdash_{cpl} b \quad R, f : \varphi, X \vdash t : g}{R, h : (a \rightarrow b) \rightarrow c, X \vdash t[f := s] : g}^{cpl_1}$$

Левая посылка не содержит λ -терма — только классическое свидетельство A' . Терм s синтезируется из переменной h импликационной клаузы и существования $p : b$. Правило (cpl_1) — линейная замена связки $(ljt) + (cut_{ipl})$: вместо явного IPL-вывода b слева используется классическая проверка, а подстановка $t[f := s]$ выполняет сечение с выученной клаузой φ .

Пример. Пусть $A = \{a\}$, тогда $A \setminus \{a\} = \emptyset$ и $\varphi = c$. Если $R, \{a\} \vdash_{cpl} b$ и $p : b$ при $x : a$, то $s = h(\lambda x^a. p) : c$, и заключение имеет вид $t[f := h(\lambda x^a. p)] : g$.

Процедура поиска вывода

Процедура $\text{proveR}(R, X, g)$ реализует поиск вывода в $C^{\rightarrow}$ с помощью двух вложенных циклов и инкрементального SAT-решателя s .

Главный цикл начинается с пустого множества интерпретаций W и вызова SAT-решателя: $R(s) \vdash_{cpl} g$? Если ответ положительный, секвенция выводима

по правилу (cpl_0). В противном случае SAT-решатель возвращает классическую контрмодель M , и начинается внутренний цикл.

Внутренний цикл пополняет множество W интерпретациями и ищет пару $\langle w, \lambda \rangle$, где $w \in W$, $\lambda = (a \rightarrow b) \rightarrow c \in X$ и w не реализует λ в модели $K(W)$. Если такой пары нет, $K(W)$ является контрмоделью Крипке и процедура завершается с ответом CountSat. Если пара найдена, SAT-решателю передаётся запрос: $R(s), w \cup \{a\} \vdash_{cpl} b$? При отрицательном ответе возвращённая интерпретация M добавляется в W , и внутренний цикл продолжается. При положительном ответе из возвращённых допущений A формируется выученная клауза $\varphi = \bigwedge(A \setminus \{a\}) \rightarrow c$, которая добавляется в SAT-решатель, множество W очищается и главный цикл перезапускается (restart).

Завершаемость главного цикла гарантируется тем, что выученные клаузы попарно не классически эквивалентны: каждая новая клауза φ_k ложна на интерпретации w_k , а все ранее выученные — истинны на ней. Завершаемость внутреннего цикла следует из строгого роста W на каждой итерации при конечном универсуме $U(s)$.

Преимущества перед LJT_{SAT}

Исчисление $C^{\rightarrow}$ и процедура proveR обладают рядом преимуществ по сравнению с LJT_{SAT} и исходной процедурой intuit [1].

- **Линейная структура вывода.** Вывод в $C^{\rightarrow}$ представляет собой единственную ветвь без сечений, тогда как вывод в LJT_{SAT} может содержать ветвления (правило (ljt)) и применения правила сечения. Это упрощает извлечение λ -терма.
- **Компактные контрмодели.** Перезапуск (restart) очищает множество W , исключая избыточные миры. На бенчмарке SYJ212 ($k = 25$) контрмодель intuitR содержит 4 мира против 1955 у intuit.
- **Производительность.** На стандартном наборе из 1200 бенчмарков intuitR решает больше задач и в целом работает быстрее, чем intuit, fCube и intHistGC.

ЗАКЛЮЧЕНИЕ

В данной работе рассмотрена задача верификации населённости полиморфных функциональных типов с позиции теории доказательств интуиционистской логики.

В исследовательской части изложены теоретические основы: интуиционистская логика и ВНК-интерпретация, семантика Крипке, системы натурального вывода и секвенциального исчисления LJ, а также изоморфизм Карри–Говарда, устанавливающий соответствие между доказательствами и типизированными λ -термами. Показано, как правила секвенциального исчисления аннотируются λ -термами для импликативного фрагмента и для полного пропозиционального языка с конъюнкцией, дизъюнкцией и абсурдом.

В конструкторской части описаны исчисления, пригодные для автоматического поиска вывода:

- исчисление LJТ (G4ip) Дайкхоффа, в котором замена правила $(\rightarrow L)$ четырьмя специализированными правилами гарантирует завершаемость обратного поиска вывода без явного контроля циклов;
- исчисление LJT_{SAT} , объединяющее поиск вывода в LJТ с инкрементальным SAT-решателем по схеме CEGAR и предоставляющее два исхода: вывод либо контрмодель Крипке;
- аннотирование правил LJТ и правил клаузификации λ -термами, что позволяет по найденному выводу извлечь свидетель населённости типа;
- исчисление $C^{\rightarrow}$, предложенное Фиорентини, в котором выводы имеют линейную структуру без сечений, а процедура proveR с механизмом перезапуска обеспечивает более высокую производительность и компактные контрмодели по сравнению с LJT_{SAT} .

Показано, что процедура клаузификации, преобразующая произвольную формулу в клаузальную форму, может быть представлена как последовательность допустимых правил секвенциального исчисления и аннотирована термами. Описан механизм извлечения λ -терма из клаузальной формы с использованием равномерной подстановки и β -редукции.

СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ

1. Claessen K., Rosén D. SAT modulo Intuitionistic Implications // Logic for Programming, Artificial Intelligence, and Reasoning (LPAR). T. 9450. — Springer, 2015. — С. 622—637. — (Lecture Notes in Computer Science).
2. Fiorentini C., Goré R., Graham-Lengrand S. A proof-theoretic perspective on SMT-solving for intuitionistic propositional logic // Automated Reasoning with Analytic Tableaux and Related Methods (TABLEAUX). T. 11714. — Springer, 2019. — С. 111—129. — (Lecture Notes in Computer Science).
3. Dyckhoff R. Contraction-free sequent calculi for intuitionistic logic // Journal of Symbolic Logic. — 1992. — Т. 57, № 3. — С. 795—807.
4. Fiorentini C. Efficient SAT-based Proof Search in Intuitionistic Propositional Logic // Automated Deduction (CADE). T. 12699. — Springer, 2021. — С. 217—233. — (Lecture Notes in Computer Science).

ПРИЛОЖЕНИЕ А

Листинг 1 Обратный поиск вывода в LJТ

```
1: function LJTProve( $\Gamma$ ,  $G$ )
2:   if  $G$  — атом и  $G \in \Gamma$  then return true
3:   if  $\perp \in \Gamma$  then return true
4:   if  $\exists A \wedge B \in \Gamma$  then
5:      $\Gamma' \leftarrow \Gamma \setminus \{A \wedge B\}$  return LJTProve( $\Gamma' \cup \{A, B\}$ ,  $G$ )
6:   if  $\exists A \vee B \in \Gamma$  then
7:      $\Gamma' \leftarrow \Gamma \setminus \{A \vee B\}$  return LJTProve( $\Gamma' \cup \{A\}$ ,  $G$ )  $\wedge$  LJTProve( $\Gamma' \cup \{B\}$ ,  $G$ )
8:   if  $\exists p \rightarrow B \in \Gamma$ ,  $p$  — атом,  $p \in \Gamma$  then return LJTProve( $(\Gamma \setminus \{p \rightarrow$ 
    $B\}) \cup \{B\}$ ,  $G$ )
9:   if  $\exists (A \wedge B) \rightarrow C \in \Gamma$  then
10:     $\Gamma' \leftarrow \Gamma \setminus \{(A \wedge B) \rightarrow C\}$  return LJTProve( $\Gamma' \cup \{A \rightarrow (B \rightarrow C)\}$ ,  $G$ )
11:   if  $\exists (A \vee B) \rightarrow C \in \Gamma$  then
12:     $\Gamma' \leftarrow \Gamma \setminus \{(A \vee B) \rightarrow C\}$  return LJTProve( $\Gamma' \cup \{A \rightarrow C, B \rightarrow C\}$ ,  $G$ )
13:   if  $G = A \rightarrow B$  then return LJTProve( $\Gamma \cup \{A\}$ ,  $B$ )
14:   if  $G = A \wedge B$  then return LJTProve( $\Gamma$ ,  $A$ )  $\wedge$  LJTProve( $\Gamma$ ,  $B$ )
15:   if  $G = A \vee B$  then
16:     if LJTProve( $\Gamma$ ,  $A$ ) then return true
17:     if LJTProve( $\Gamma$ ,  $B$ ) then return true
18:   for  $(C_1 \rightarrow C_2) \rightarrow B \in \Gamma$  do
19:      $\Gamma' \leftarrow \Gamma \setminus \{(C_1 \rightarrow C_2) \rightarrow B\}$ 
20:     if LJTProve( $\Gamma' \cup \{C_2 \rightarrow B\}$ ,  $C_1 \rightarrow C_2$ ) then return LJTProve( $\Gamma' \cup$ 
    $\{B\}$ ,  $G$ )
21:   return false
```

ПРИЛОЖЕНИЕ Б

Вывод формулы $((((a \wedge b) \rightarrow c) \rightarrow ((a \rightarrow c) \vee (b \rightarrow c))) \rightarrow c) \rightarrow c$ в исчислении LJТ.

Для компактности обозначим $\alpha = (a \wedge b) \rightarrow c$ и $\beta = (a \rightarrow c) \vee (b \rightarrow c)$. Тогда доказываемая формула принимает вид $((\alpha \rightarrow \beta) \rightarrow c) \rightarrow c$.

$$\begin{array}{c}
 \frac{}{x:(a \rightarrow c) \rightarrow c, y:b \rightarrow c, z:a \vdash y : b \rightarrow c} Ax \\
 \frac{}{x:(a \rightarrow c) \rightarrow c, m:a \rightarrow (b \rightarrow c), z:a \vdash mz : b \rightarrow c} \rightarrow L_1 \quad \frac{}{x:(a \rightarrow c) \rightarrow c, k:c, m:a \rightarrow (b \rightarrow c) \vdash k : c} Ax \\
 \frac{}{x:(a \rightarrow c) \rightarrow c, p:(b \rightarrow c) \rightarrow c, m:a \rightarrow (b \rightarrow c), z:a \vdash p(mz) : c} \rightarrow R \\
 \frac{}{x:(a \rightarrow c) \rightarrow c, p:(b \rightarrow c) \rightarrow c, m:a \rightarrow (b \rightarrow c) \vdash \lambda z. p(mz) : a \rightarrow c} \rightarrow L_2 \\
 \frac{}{x:(a \rightarrow c) \rightarrow c, p:(b \rightarrow c) \rightarrow c, q:\alpha \vdash \lambda z. p(\lambda w. q\langle z, w \rangle) : a \rightarrow c} \rightarrow L_3 \\
 \frac{}{s:\beta \rightarrow c, q:\alpha \vdash \lambda z. (\lambda u. s(Ru))(\lambda w. q\langle z, w \rangle) : a \rightarrow c} \vee R_1 \\
 \frac{}{s:\beta \rightarrow c, q:\alpha \vdash L(\lambda z. (\lambda u. s(Ru))(\lambda w. q\langle z, w \rangle)) : \beta} \rightarrow R \\
 \frac{}{s:\beta \rightarrow c \vdash \lambda q. L(\lambda z. (\lambda u. s(Ru))(\lambda w. q\langle z, w \rangle)) : \alpha \rightarrow \beta} \rightarrow L_4 \quad \frac{}{k:c \vdash k : c} Ax \\
 \frac{}{r:(\alpha \rightarrow \beta) \rightarrow c \vdash r(\lambda q. L(\lambda z. (\lambda u. (\lambda t. r(Kt))(Ru))(\lambda w. q\langle z, w \rangle))) : c} \rightarrow R \\
 \vdash \lambda r. r(\lambda q. L(\lambda z. (\lambda u. (\lambda t. r(Kt))(Ru))(\lambda w. q\langle z, w \rangle))) : ((\alpha \rightarrow \beta) \rightarrow c) \rightarrow c
 \end{array}$$

ПРИЛОЖЕНИЕ В

Ниже приведены процедуры поиска вывода в исчислении LJT_{SAT} , основанные на работе Фиорентини, Горе и Грэма-Ленгрена. Процедуры используют инкрементальный SAT-решатель s с интерфейсом:

- $\text{newSolver}()$ — создать новый SAT-решатель;
- $\text{addClause}(s, \varphi)$ — добавить плоскую клаузу φ в решатель s ;
- $\text{satProve}(s, A, q)$ — проверить, выполняется ли $R(s), A \vdash_{cpl} q$, где $R(s)$ — множество клауз в s , A — множество атомарных допущений. Возвращает $\text{Yes}(A')$ с $A' \subseteq A$ или $\text{No}(M)$ с классической контрмоделью M .

Листинг 2 Главная процедура LJT_{SAT}

```
1: function LJTSatMain( $R_0, X_0, q_0$ )
2:    $s \leftarrow \text{newSolver}()$ 
3:   for  $\varphi \in R_0$  do
4:      $\text{addClause}(s, \varphi)$ 
5:   for  $(a \rightarrow b) \rightarrow c \in X_0$  do
6:      $\text{addClause}(s, b \rightarrow c)$ 
7:    $\tau \leftarrow \text{LJTSat}(R_0, X_0, \emptyset, q_0)$ 
8:   if  $\tau = \text{No}(K)$  then
9:     return  $K$ 
10:  else
11:    return ВЫВОД в  $LJT_{SAT}$ 
```

Листинг 3 Рекурсивная процедура поиска вывода

```
1: function LJTSat( $R, X, A, q$ )
2:    $\tau_0 \leftarrow \text{satProve}(s, A, q)$ 
3:   if  $\tau_0 = \text{Yes}(A')$  then
4:     return  $\text{Yes}(A')$ 
5:    $M \leftarrow$  контрмодель из  $\tau_0$ 
6:    $\Theta \leftarrow \emptyset$ 
7:   for  $\iota = (a \rightarrow b) \rightarrow c \in X$  т. ч.  $a \notin M, b \notin M, c \notin M$  do
8:      $X_\iota \leftarrow X \setminus \{\iota\}$ 
9:      $\tau_1 \leftarrow \text{LJTSat}(R \cup \{b \rightarrow c\}, X_\iota, M \cup \{a\}, b)$ 
10:    if  $\tau_1 = \text{No}(K_1)$  then
11:       $\Theta \leftarrow \Theta \cup \{(\iota, K_1)\}$ 
12:    else
13:       $A_1 \leftarrow$  множество атомов из  $\tau_1$ 
14:       $\tilde{\varphi} \leftarrow \bigwedge(A_1 \setminus \{a\}) \rightarrow c$ 
15:       $\text{addClause}(s, \tilde{\varphi})$ 
16:       $\tau_2 \leftarrow \text{LJTSat}(R \cup \{\tilde{\varphi}\}, X, A, q)$ 
17:      if  $\tau_2 = \text{No}(K_2)$  then
18:        return  $\text{No}(K_2)$ 
19:      else
20:        return  $\tau_2$ 
21:  return  $\text{No}(\text{Mod}(r, \Theta, M))$ 
```

Параметр r (мир контрмодели) опущен для краткости; при построении контрмодели миры индексируются последовательностями импликационных клаузов, образующими дерево вызовов.